



Construcción de software y toma de decisiones (Gpo 501)

**Competencia STC204 Desarrollo de
Componentes de Software**

Carlos Arturo Gómez Ayala - A01711027
ITESM Campus Querétaro
Profesor: Enrique Alfonso Calderón Balderas
Co-Titular: Alejandro Fernández Vilchis
Co-Titular: Denisse Lizbeth Maldonado Flores

Fecha: 3/06/2026

Introducción

Este documento presenta mi competencia y mi participación individual en el desarrollo del sistema CoreData a lo largo de esta Unidad de formación, una plataforma web de cumplimiento PLD/FT (Prevención de Lavado de Dinero y Financiamiento al Terrorismo) para SVA LAW, una SOFOM regulada por la CNBV.

El sistema fue construido en equipo con arquitectura MVC usando Node.js + Express + EJS + Supabase, que conecta un tanto con la competencia de implantación de software. Mi contribución específica fue desde un inicio en documentación clave e importante con cada avance semanal, en generación de distintos diagramas como Mer, en participación en el equipo y en exposiciones. Al momento del diseño del código desde el 0% hasta del Avance del 70%, 90% y versión final 1.0 del diseño e implementación de requisitos funcionales y documentación. Y del diseño y ejecución de pruebas después del 30% en adelante. Trabajo en equipo realizado donde contribuimos semanalmente con compromiso y dedicación, con constante comunicación por medio de reuniones y de la plataforma social de Whatsapp, del repositorio oficial y de alternativos para distintas pruebas y códigos con la finalidad de afinar el producto a cada entrega.

El código mostrado será en color azul, será mostrado completo o en fragmentos. Esta metodología también se llevó a cabo para distintas entregas de Laboratorios. No se usarán líneas específicas pues puede añadirse o eliminarse código para hacer el sistema más eficiente, se pondrá la ruta donde debería estar esos segmentos hasta el día de hoy.

Competencia desarrollada

STC0204: Desarrolla los componentes diseñados de un sistema computacional. Conoce convenciones de código y codifica aplicando una convención de código en la aplicación web y en los scripts de la base de datos. Desarrolla el front-end apegado a los diseños de componentes, así como a las heurísticas de usabilidad. Desarrolla el back-end apegado a los diseños de componentes. Codifica interacciones asíncronas cliente-servidor. Integra servicios web en sus aplicaciones. Participa activa y sustancialmente en la codificación del proyecto desde el inicio de la codificación del proyecto. Integra de manera disciplinada y frecuente su código en la rama de desarrollo del proyecto desde el inicio de la codificación del proyecto. Existe evidencia de la práctica de pair-programming al menos en la mitad de los laboratorios y trabajo en el proyecto.

Para poder alcanzar el nivel Sólido o Destacado en esta competencia y dejando en demostración mis contribuciones cubren:

- La Implementación de componentes de back-end (middleware, controllers, models, routes)
- Implementación de componentes de front-end (vistas EJS, lógica JS del cliente)
- Aplicación de convenciones de código documentadas dentro y fuera del código.
- Integración al repositorio del equipo con commits
- integración de Supabase como servicio web externo

RBAC — Control de acceso basado en roles

Se trabajó en distintos archivos de frontend y backend. Algunos de estos siendo los siguientes y varios más que se detallarán en el documento.

Backend siendo el servidor — Node.js:

middleware/auth.js — lógica del servidor, verifica rol antes de ejecutar o correr cualquier cosa.

controllers/login.controller.js — procesa el login, crea la sesión.

controllers/portal.controller.js — consulta Supabase y prepara los datos.

routes/portal.routes.js — define las rutas del servidor.

index.js — aplica el RBAC a nivel de servidor en cada grupo de rutas y el sistema.

Frontend de cliente — navegador:

views/nav.ejs — HTML que ve el usuario, menú condicional por rol.

public/js/db-ajax.js — JavaScript que corre en el navegador, oculta el botón de resolver para Analista.

Módulos implementados

A continuación, se describen los cinco módulos que desarrollé con el equipo con el código exacto, las rutas de cada archivo, los números de línea para tomar screenshots y la explicación de qué hace cada uno.

RBAC Control de acceso basado en roles

RF cubierto: RNF-01 Seguridad de acceso

Antes de la implementación propuesta aquí, el sistema solo tenía un middleware básico que verificaba si había sesión, pero cualquier usuario podía entrar a cualquier ruta escribiéndola en la URL y junto a las credenciales demo aplicadas en principios del código, un login que fue pionero para nuestro sistema en index.js. No había distinción entre roles. Yo diseñé e implementé el sistema completo de control de acceso por rol (RBAC), con el apoyo y comprensión de distintos recursos. Que también se presentó en distintas partes como conocimiento severo, por ejemplo, en la competencia de implantación de software.

Ruta del código inicial para middleware/auth.js

```
//No moverle son los roles
//Autenticacion y control de acceso por rol (RBAC)
// verificarSesion bloquea las rutas si no hay 1 sesion activa
//verificarRol: bloquea rutas si el rol del usuario no esta permitido
function verificarSesion(req, res, next) {
  if (req.session && req.session.usuario) {
    return next();
  }
  res.redirect('/login?mensaje=Debes iniciar sesion para continuar.');
```



```
}

// Uso para poder verificarRol('Oficial de Cumplimiento', 'Analista')
function verificarRol(...roles) {
  return function(req, res, next) {
    if (!req.session || !req.session.usuario) {
      return res.redirect('/login?mensaje=Debes iniciar sesion para continuar.');
```



```
    }
    if (roles.includes(req.session.usuario.rol)) {
      return next();
    }
    // Redirigir segun el rol para que no se muestre 1 pantalla en blanco
    //esto despues puede ser un test
    const rol = req.session.usuario.rol;
    if (rol === 'Cliente') return res.redirect('/portal/mi-expediente?mensaje=Sin
acceso.');
```

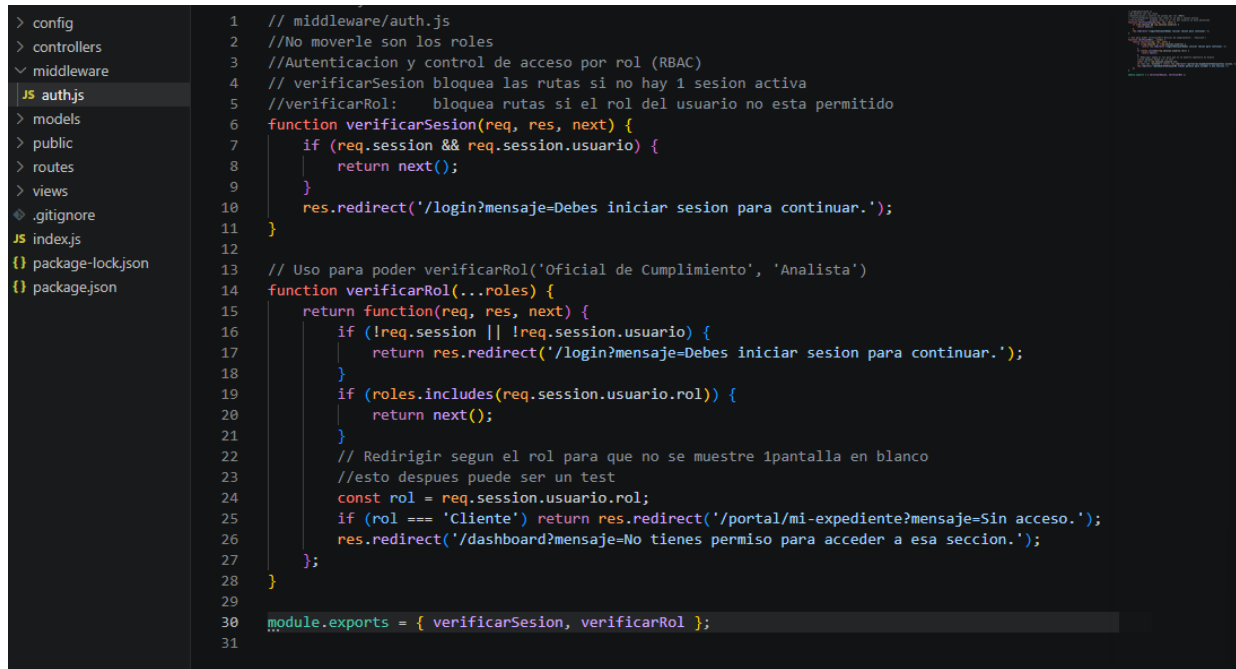


```
    res.redirect('/dashboard?mensaje=No tienes permiso para acceder a esa
seccion.');
```



```
  };
}
```

```
module.exports = { verificarSesion, verificarRol };
```



```
> config
> controllers
> middleware
  JS auth.js
> models
> public
> routes
> views
> .gitignore
JS index.js
() package-lock.json
() package.json

1 // middleware/auth.js
2 //No moverle son los roles
3 //Autenticacion y control de acceso por rol (RBAC)
4 // verificarSesion bloquea las rutas si no hay 1 sesion activa
5 //verificarRol: bloquea rutas si el rol del usuario no esta permitido
6 function verificarSesion(req, res, next) {
7   if (req.session && req.session.usuario) {
8     return next();
9   }
10  res.redirect('/login?mensaje=Debes iniciar sesion para continuar.');
```

Para el RBAC era necesaria una función identificadora por lo que se utilizó la llamada “verificarRol”, agregados en distintos fragmentos del index.js.

Líneas iniciales

```
// Middlewares RBAC para accesos y seguridad
```

```
const { verificarSesion, verificarRol } = require('./middleware/auth');
```

```
// Modelos para el dashboard
```

```
const modelClientes = require('./models/clientes.model');
```

```
const modelOperaciones = require('./models/operaciones.model');
```

```
const modelAlertas = require('./models/alertas.model');
```

Saltando líneas, fragmento:

```
//PORTAL CLIENTE solo Cliente
```

```
const rutasPortal = require('./routes/portal.routes');
```

```
app.use('/portal', verificarSesion, verificarRol('Cliente'), rutasPortal);
```

```
//DASHBOARD del Oficial + Analista
```

```
app.get('/dashboard',
```

```
  verificarSesion,
```

```
  verificarRol('Oficial de Cumplimiento', 'Analista'),
```

```
  (req, res) => res.render('dashboard', { mensaje: req.query.mensaje || null })
```

```
);
```

```
app.get('/api/dashboard',  
  verificarSesion,  
  verificarRol('Oficial de Cumplimiento', 'Analista'),  
  async (req, res) => {...  
  ...
```

```
//CLIENTES Oficial + Analista
```

```
const rutasClientes = require('./routes/clientes.routes');  
app.use('/clientes', verificarSesion, verificarRol('Oficial de Cumplimiento',  
'Analista'), rutasClientes);
```

```
//OPERACIONES Oficial + Analista
```

```
const rutasOperaciones = require('./routes/operaciones.routes');  
app.use('/operaciones', verificarSesion, verificarRol('Oficial de Cumplimiento',  
'Analista'), rutasOperaciones); ...
```

```
COREDATAFINAL  
> config  
> controllers  
▼ middleware  
JS authjs  
> models  
> public  
> routes  
> views  
◆ .gitignore  
JS indexjs  
{ package-lock.json  
{ package.json  
JS indexjs > ...  
92 );  
93  
94 // Al colocar oficial, se entiende que tiene un grado alto de autoridad y de acceso al sistema  
95 //sin embargo tambien el sistema deberia ser accesible a mandos altos del cliente de swa law o administradores.  
96 //  
97  
98 //CLIENTES Oficial + Analista  
99 const rutasClientes = require('./routes/clientes.routes');  
100 app.use('/clientes', verificarSesion, verificarRol('Oficial de Cumplimiento', 'Analista'), rutasClientes);  
101  
102 //OPERACIONES Oficial + Analista  
103 const rutasOperaciones = require('./routes/operaciones.routes');  
104 app.use('/operaciones', verificarSesion, verificarRol('Oficial de Cumplimiento', 'Analista'), rutasOperaciones);  
105  
106 //ALERTAS Oficial + Analista  
107 const rutasAlertas = require('./routes/alertas.routes');  
108 app.use('/alertas', verificarSesion, verificarRol('Oficial de Cumplimiento', 'Analista'), rutasAlertas);  
109  
110 //CONTRATOS Oficial + Analista  
111 const rutasContratos = require('./routes/contratos.routes');  
112 app.use('/contratos', verificarSesion, verificarRol('Oficial de Cumplimiento', 'Analista'), rutasContratos);  
113  
114 //BUZON INTERNO Oficial + Analista  
115 const rutasBuzonInterno = require('./routes/buzon_interno.routes');  
116 app.use('/buzon-interno', verificarSesion, verificarRol('Oficial de Cumplimiento', 'Analista'), rutasBuzonInterno);  
117  
118 // REPORTE solo Oficial  
119 const rutasReportes = require('./routes/reportes.routes');  
120 app.use('/reportes', verificarSesion, verificarRol('Oficial de Cumplimiento'), rutasReportes);  
121  
122 //ADMIN solo Oficial y en teoria admin  
123 const rutasAdmin = require('./routes/admin.routes');  
124 app.use('/admin', verificarSesion, verificarRol('Oficial de Cumplimiento'), rutasAdmin);  
125  
126 //REGLAS solo Oficial, admin o la sofom un mando alto  
127 const rutasReglas = require('./routes/reglas.routes');  
128 app.use('/reglas', verificarSesion, verificarRol('Oficial de Cumplimiento'), rutasReglas);  
129  
130 //HISTORIAL solo Oficial  
131 const rutasHistorial = require('./routes/historial.routes');  
132 app.use('/historial', verificarSesion, verificarRol('Oficial de Cumplimiento'), rutasHistorial);  
133
```

Además de documentar autoría, apliqué comentarios que explican el porqué de cada decisión técnica. Esto sucede en muchas partes del código de parte mía y de todos en el equipo CoreData.

Continuando en el mismo, para tener una idea nos pensamos dentro del MVC, pasamos a controllers. controllers/login.controller.js

```
const modelLogin = require('../models/login.model');
const supabase = require('../config/supabase');

// Usuarios demo hardcoded para pruebas
const usuariosDemo = {
  'demo@sofom.mx': {
    nombre: 'Usuario Demo', correo: 'demo@sofom.mx',
    password: 'demo123', rol: 'Oficial de Cumplimiento', idcliente: null
  },
  'analista@sofom.mx': {
    nombre: 'Carlos Analista', correo: 'analista@sofom.mx',
    password: 'analista123', rol: 'Analista', idcliente: null
  },
  'juan.perez@email.com': {
    nombre: 'Juan Perez', correo: 'juan.perez@email.com',
    password: 'cliente123', rol: 'Cliente', idcliente: null
  }
};...
```

Estas credenciales fueron guardadas en el repositorio propio, del repositorio de equipo y en evidencia para la competencia en caso de que se corra el sistema.

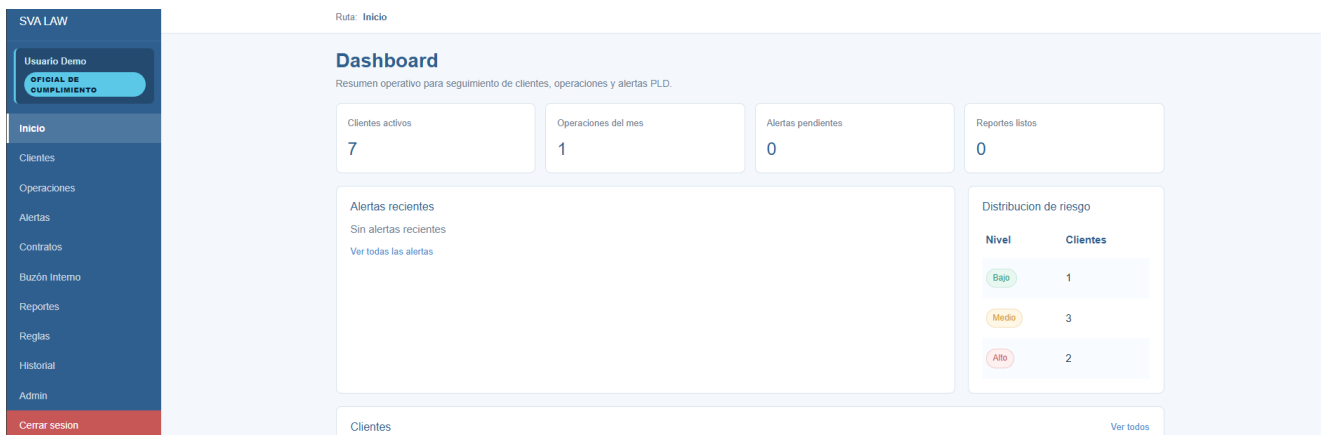
```
EXPLORER    ...    JS authjs    JS index.js    JS login.controller.js X
COREDATAFINAL
  > config
  > controllers
    JS admin.controller.js
    JS alertas.controller.js
    JS buzon_interno.contr...
    JS clientes.controller.js
    JS contratos.controller.js
    JS historial.controller.js
    JS login.controller.js
    JS operaciones.controll...
    JS portal.controller.js
    JS reglas.controller.js
    JS reportes.controller.js
  > middleware
    JS authjs
    > models
    > public
    > routes
    > views
    > .gitignore
    JS index.js
    {} package-lock.json
    {} package.json
  > OUTLINE
  > TIMELINE

controllers > JS login.controller.js > ProcesarLogin > ProcesarLogin
1 //Autenticación hasta el momento de creación con 3 roles:
2 // Oficial de Cumplimiento, Analista, Cliente
3
4 const modelLogin = require('../models/login.model');
5 const supabase = require('../config/supabase');
6
7 // Usuarios demo hardcoded para pruebas
8 const usuariosDemo = {
9   'demo@sofom.mx': {
10     nombre: 'Usuario Demo', correo: 'demo@sofom.mx',
11     password: 'demo123', rol: 'Oficial de Cumplimiento', idcliente: null
12   },
13   'analista@sofom.mx': {
14     nombre: 'Carlos Analista', correo: 'analista@sofom.mx',
15     password: 'analista123', rol: 'Analista', idcliente: null
16   },
17   'juan.perez@email.com': {
18     nombre: 'Juan Perez', correo: 'juan.perez@email.com',
19     password: 'cliente123', rol: 'Cliente', idcliente: null
20   }
21 };
22
23 module.exports.VistaLogin = async (req, res) => {
24   if (req.session && req.session.usuario) {
25     if (req.session.usuario.rol === 'Cliente') return res.redirect('/portal/mi-expediente');
26     return res.redirect('/dashboard');
27   }
28   res.render('../login/login', {
29     usuarioDemo: { correo: 'demo@sofom.mx', password: 'demo123' },
30     mensaje: req.query.mensaje || null
31   });
32 };
33
34 module.exports.ProcesarLogin = async (req, res) => {
35   const { correo, password } = req.body;
36
37   // 1. Revisar los usuarios demo de los roles
38   const demo = usuariosDemo[correo];
39   if (demo && demo.password === password) {
40     req.session.usuario = {
41       id: 0, nombre: demo.nombre, correo: demo.correo,
42       rol: demo.rol, idcliente: demo.idcliente
43     };
44     if (demo.rol === 'Cliente') return res.redirect('/portal/mi-expediente');
45     return res.redirect('/dashboard');
46   }
47
48   // 2. Validar con la BSupabase
```

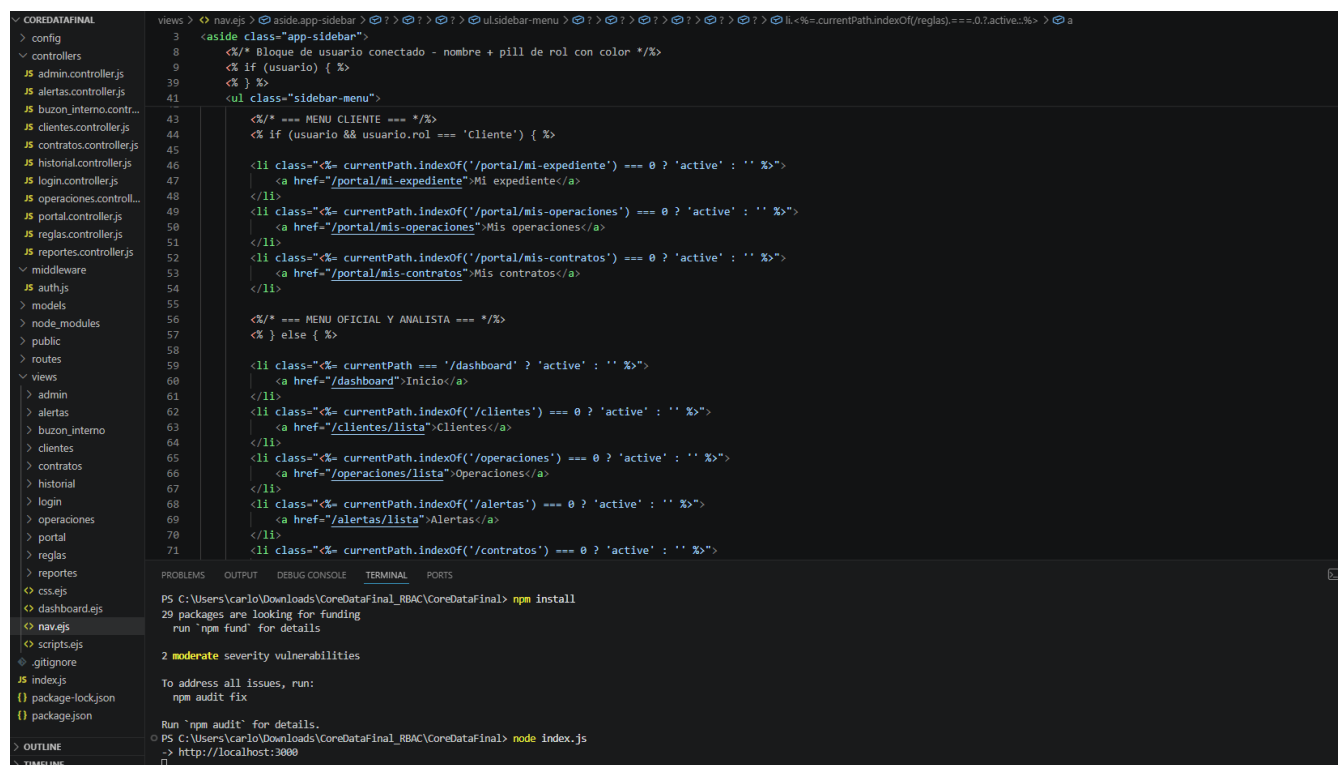
Navegacion con rol visible views/nav.ejs

RF cubierto: RNF-01 + Usabilidad

Lo que se hizo fue una reescritura y revisión completamente el nav.ejs para que muestre el nombre y rol del usuario conectado con colores que contrasten sobre el sidebar azul y el menu sea diferente según el rol. Teniendo mejores views para cliente o usuarios.



...



Portal del cliente

RF cubierto: RF-02 Consultar expediente, RF-06 Ver operaciones, RF-14 Ver contratos propios

Esta sección sigue en desarrollo y a discusión si es el camino correcto para seguir, pero por el momento es funcional y da las rutas y vistas. Este módulo no existía en el proyecto. Se creo con rutas, controlador y las 3 vistas. El punto hasta el momento es que el cliente solo puede ver su propia información.

Views/portal/mi_expediente.ejs

```
<!DOCTYPE html>
<html lang="es">
<head><%- include('../css') %></head>
<body>
<%- include('../nav') %>
<main class="app-page">
  <div class="app-header">
    <div>
      <h1 class="app-title">Mi Expediente</h1>
      <p class="app-subtitle">Consulta tu informacion registrada en el
sistema.</p>
    </div>
  </div>

  <% if (mensaje) { %>
```

```
...
EXPLORER  JS authjs  JS indexjs  JS login.controller.js  mi_expediente.ejs X
views > portal > mi_expediente.ejs > html > body > ? > main-app-page > ?
1 <!DOCTYPE html>
2 <html lang="es">
3 <head><%- include('../css') %></head>
4 <body>
5 <%- include('../nav') %>
6 <main class="app-page">
7   <div class="app-header">
8     <div>
9       <h1 class="app-title">Mi Expediente</h1>
10      <p class="app-subtitle">Consulta tu informacion registrada en el sistema.</p>
11    </div>
12  </div>
13
14  <% if (mensaje) { %>
15    <div class="alert-msg alert-warning" style="background: #fff3cd;border:1px solid #ffe107;padding:12px 16px;border-radius:6px;margin-bottom:16px;color: #856404;">
16      <%= mensaje %>
17    </div>
18  <% } %>
19
20  <% if (cliente) { %>
21    <section class="panel">
22      <div class="detail-grid" style="display:grid;grid-template-columns:repeat(auto-fill,minmax(220px,1fr));gap:16px;">
23        <div class="detail-item">
24          <span style="display:block;font-size:12px;color: #888;margin-bottom:4px;">Nombre / Razon Social</span>
25          <span style="font-weight:600;"><%= cliente.nombrerazonsocial || '-' %></span>
26        </div>
27        <div class="detail-item">
28          <span style="display:block;font-size:12px;color: #888;margin-bottom:4px;">RFC</span>
29          <span style="font-weight:600;"><%= cliente.rfc || '-' %></span>
30        </div>
31        <div class="detail-item">
32          <span style="display:block;font-size:12px;color: #888;margin-bottom:4px;">CURP</span>
33          <span style="font-weight:600;"><%= cliente.curp || '-' %></span>
34        </div>
35        <div class="detail-item">
36          <span style="display:block;font-size:12px;color: #888;margin-bottom:4px;">Tipo de cliente</span>
37          <span style="font-weight:600;"><%= cliente.tipocliente || '-' %></span>
38        </div>
39        <div class="detail-item">
40          <span style="display:block;font-size:12px;color: #888;margin-bottom:4px;">Correo electronico</span>
41          <span style="font-weight:600;"><%= cliente.correoelectronico || '-' %></span>
42        </div>
43      </div>
```

Views/portal/mis_contratos.ejs

```
<!DOCTYPE html>
<html lang="es">
<head><%- include('../css') %></head>
<body>
<%- include('../nav') %>
<main class="app-page">
  <div class="app-header">
    <div>
      <h1 class="app-title">Mis Contratos</h1>
      <p class="app-subtitle">Contratos activos vinculados a tu cuenta.</p>
    </div>
  </div>
  <section class="panel">...
```

Views/portal/mis_operaciones.ejs

```
<!DOCTYPE html>
<html lang="es">
<head><%- include('../css') %></head>
<body>
<%- include('../nav') %>
<main class="app-page">
  <div class="app-header">
    <div>
      <h1 class="app-title">Mis Operaciones</h1>
      <p class="app-subtitle">Historial de tus operaciones registradas.</p>
    </div>
  </div>
  <section class="panel">
```

```
EXPLORER    ...    JS auth.js    JS index.js    JS login.controller.js    mis_operaciones.ejs X
COREDATAFINAL
  > config
  > controllers
  > middleware
  JS auth.js
  > models
  > node_modules
  > public
  > routes
  > views
    > admin
    > alertas
    > buzon_interno
    > clientes
    > contratos
    > historial
    > login
    > operaciones
    > portal
      < mis_operaciones.ejs
      < mi_expediente.ejs
      < mis_contratos.ejs
    > reglas
    > reportes
    < css.ejs
    < dashboard.ejs
    < nav.ejs
    < scripts.ejs
    < .gitignore
  JS index.js
  {} package-lock.json
  {} package.json

views > portal > mis_operaciones.ejs > html > body > ? > ?
1  <!DOCTYPE html>
2  <html lang="es">
3  <head><%- include('../css') %></head>
4  <body>
5  <%- include('../nav') %>
6  <main class="app-page">
7    <div class="app-header">
8      <div>
9        <h1 class="app-title">Mis Operaciones</h1>
10       <p class="app-subtitle">Historial de tus operaciones registradas.</p>
11     </div>
12   </div>
13   <section class="panel">
14     <% if (operaciones && operaciones.length > 0) { %>
15     <table class="data-table">
16       <thead>
17         <tr><th>ID</th><th>Tipo</th><th>Producto</th><th>Monto</th><th>Moneda</th><th>Fecha</th><th>Estatus</th></tr>
18       </thead>
19       <tbody>
20         <% operaciones.forEach(op => { %>
21           <tr>
22             <td><%= op.idoperacion %></td>
23             <td><%= op.tipooperacion || '-' %></td>
24             <td><%= op.producto || '-' %></td>
25             <td><%= Number(op.monto||0).toLocaleString('es-MX') %></td>
26             <td><%= op.moneda || 'MXN' %></td>
27             <td><%= op.fecha ? String(op.fecha).substring(0,10) : '-' %></td>
28             <td><span class="badge badge-info"><%= op.estatus || 'Registrada' %></span></td>
29           </tr>
30         <% }); %>
31       </tbody>
32     </table>
33     <% } else { %>
34       <p style="color:#888;padding:16px 0;">No hay operaciones registradas aun.</p>
35     <% } %>
36   </section>
37 </main>
38 <%- include('../scripts') %>
39 </body></html>
40
```

Routes/portal.routes.js

// routes/portal.routes.js

// Rutas exclusivas para el rol del Cliente

const express = require('express');

const router = express.Router();

const controller = require('../controllers/portal.controller');

router.get('/mi-expediente', controller.MiExpediente);

router.get('/mis-operaciones', controller.MisOperaciones);

router.get('/mis-contratos', controller.MisContratos);

module.exports = router;

```
▼ COREDATAFINAL
  ▼ middleware
  > models
  > node_modules
  > public
  ▼ routes
    JS admin.routes.js
    JS alertas.routes.js
    JS buzon_interno.route...
    JS clientes.routes.js
    JS contratos.routes.js
    JS historial.routes.js
    JS login.routes.js
    JS operaciones.routes.js
    JS portal.routes.js
    JS reglas.routes.js
    JS reportes.routes.js

routes > JS portal.routes.js > ...
1 // routes/portal.routes.js
2 // Rutas exclusivas para el rol del Cliente
3
4
5 const express = require('express');
6 const router = express.Router();
7 const controller = require('../controllers/portal.controller');
8
9 router.get('/mi-expediente', controller.MiExpediente);
10 router.get('/mis-operaciones', controller.MisOperaciones);
11 router.get('/mis-contratos', controller.MisContratos);
12
13 module.exports = router;
14 |
```

controllers/portal.controller.js

```
const supabase = require('../config/supabase');
```

```
module.exports.MiExpediente = async (req, res) => {
  const idcliente = req.session.usuario.idcliente;
  if (!idcliente) {
    return res.render('./portal/mi_expediente', {
      cliente: null, documentos: [],
      mensaje: 'No se encontro tu expediente vinculado. Contacta a soporte.'
    });
  }
  try {
```

```
...
COREDATAFINAL
  ▼ controllers
    JS config
    JS admin.controller.js
    JS alertas.controller.js
    JS buzon_interno.contr...
    JS clientes.controller.js
    JS contratos.controller.js
    JS historial.controller.js
    JS login.controller.js
    JS operaciones.contr...
    JS portal.controller.js
    JS reglas.controller.js
    JS reportes.controller.js
  ▼ middleware
  > auth.js
  > models
  > node_modules
  > public
  ▼ routes
    JS admin.routes.js
    JS alertas.routes.js
    JS buzon_interno.route...
    JS clientes.routes.js
    JS contratos.routes.js
    JS historial.routes.js
    JS login.routes.js
    JS operaciones.routes.js
    JS portal.routes.js
    JS reglas.routes.js
    JS reportes.routes.js
  ▼ views
    > admin
    > alertas

controllers > JS portal.controller.js > ...
6 module.exports.MiExpediente = async (req, res) => {
7   const [rc, rd] = await Promise.all([
8     supabase.from('cliente').select('*').eq('idcliente', idcliente).single(),
9     supabase.from('documentocliente').select('*').eq('idcliente', idcliente)
10   ]);
11   res.render('./portal/mi_expediente', {
12     cliente: rc.data || null, documentos: rd.data || [], mensaje: null
13   });
14   catch {
15     res.render('./portal/mi_expediente', { cliente: null, documentos: [], mensaje: 'Error al cargar tu expediente. Intenta de nuevo o contacta a soporte' });
16   }
17 };
18
19 module.exports.MisOperaciones = async (req, res) => {
20   const idcliente = req.session.usuario.idcliente;
21   let operaciones = [];
22   if (idcliente) {
23     const { data } = await supabase.from('operacion').select('*')
24       .eq('idcliente', idcliente).order('idoperacion', { ascending: false });
25     operaciones = data || [];
26   }
27   res.render('./portal/mis_operaciones', { operaciones });
28 };
29
30 module.exports.MisContratos = async (req, res) => {
31   const idcliente = req.session.usuario.idcliente;
32   let contratos = [];
33   if (idcliente) {
34     const { data } = await supabase.from('contrato').select('*')
35       .eq('idcliente', idcliente).order('idcontrato', { ascending: false });
36     contratos = data || [];
37   }
38   res.render('./portal/mis_contratos', { contratos });
39 };
40
41 ...
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\carlo\Downloads\CoreDataFinal_RBAC\CoreDataFinal> npm install
PS C:\Users\carlo\Downloads\CoreDataFinal_RBAC\CoreDataFinal> node index.js
```

Validaciones de Cliente como el de RFC con regex por tipo de cliente

En anteriores entregas y avances solo se validaba que el RFC tuviera entre 12 y 13 caracteres, sin verificar el formato real. Esto permitía guardar RFCs como 'XAX1010101010' que no corresponden a ningún contribuyente real.

Ahora en el país de Mexico, se valida el formato completo con expresiones regulares distintas según el tipo de cliente seleccionado en el formulario. Caracteres reales de persona física y moral.

- Persona Física: 4 letras + 6 numeros + 3 caracteres de homoclave (13 chars total)
- Persona Moral: 3 letras + 6 numeros + 3 caracteres de homoclave (12 chars total)

[views/clientes/expediente_cliente.ejs](#).

```
function irPaso2() {
  const errores = [];

  const nombre = document.getElementById('nombre').value.trim();
  const rfc    = document.getElementById('rfc').value.trim();
  const curp   = document.getElementById('curp').value.trim();
  const tel    = document.getElementById('telefono').value.trim();
  const correo = document.getElementById('correo').value.trim();
  const cp     = document.getElementById('codigoPostal').value.trim();

  // Validacion mejorada
  // RFC persona fisica = 4letras+6nums+3homoclave (13), moral =
  3letras+6nums+3homoclave (12), poner un rcf simulado a uno real
  const rfcRegexFisica = /^[A-Z&Ñ]{4}\d{6}[A-Z0-9]{3}$/i;
  const rfcRegexMoral  = /^[A-Z&Ñ]{3}\d{6}[A-Z0-9]{3}$/i;
  const tipoCliente    = document.getElementById('tipoCliente').value;

  if (!nombre) errores.push('• El nombre / razón social es obligatorio. ');
  if (!rfc)     errores.push('• El RFC es obligatorio. ');
  if (rfc) {...
```

Correo electrónico obligatorio

[views/clientes/expediente_cliente.ejs](#)

El formulario avanzaba aunque estuviera vacío o tuviera formato invalido. Solo mostraba el error pero no bloqueaba.

Ahora el correo es obligatorio. La razón de negocio es que el sistema PLD necesita el correo para enviar notificaciones regulatorias al cliente y para identificarlo de manera unica en Supabase. Se deshabilita continuar con registro faltando alguno de los datos importantes.

views/clientes/expediente_cliente.ejs

```
// Correo es obligatorio para el sistema PLD necesario para notificaciones
if (!correo) errores.push('• El correo electrónico es obligatorio.');
```

if (correo && /^[^s@]+@[^s@]+\.[^s@]+\$/\.test(correo)) errores.push('• El correo electrónico no tiene un formato válido.');

if (tel && /^[0-9]{10}\$/\.test(tel)) errores.push('• El teléfono debe tener exactamente 10 dígitos numéricos.');

if (cp && /^[0-9]{5}\$/\.test(cp)) errores.push('• El código postal debe tener exactamente 5 dígitos numéricos.');


```
// Deshabilitar boton Siguiente mientras haya errores
// Esto evita que el usuario avance con datos invalidos en un sistema regulado por la CNBV
const btnSiguiente = document.querySelector('#paso1
.btn:not(.btn.secondary)');
if (errores.length > 0) {
  mostrarErrores(errores);
  if (btnSiguiente) btnSiguiente.disabled = true;
  return;
}
```

...

Botón siguiente deshabilitado mientras hay errores. Todas estas validaciones de información del cliente están implementados en el mismo archivos por lo que se adjunta solamente una captura de pantalla para este caso.

```

EXPLORER
COREDATAFINAL
  > config
  > controllers
  > middleware
  > models
  > node_modules
  > public
  > routes
  > views
    > admin
    > alertas
    > buzon_interno
    > clientes
      < detalle_cliente.ejs
      < documentos_client...
      < editar_cliente.ejs
      < expediente_cliente...
      < lista_clientes.ejs
    > contratos
    > historial
    > login
    > operaciones
    > portal
    > reglas
    > reportes
    < css.ejs
    < dashboard.ejs
    < nav.ejs
    < scripts.ejs
    < gitignore
  < index.js
  < package-lock.json
  < package.json

```

```

expediente_cliente.ejs
views > clientes > expediente_cliente.ejs > html > body > ? > script > irPaso2
2 <html lang="es">
6 <body>
7 <%- include('../nav.ejs') %>
195 <script>
275 function irPaso2() {
277
278     const nombre = document.getElementById('nombre').value.trim();
279     const rfc = document.getElementById('rfc').value.trim();
280     const curp = document.getElementById('curp').value.trim();
281     const tel = document.getElementById('telefono').value.trim();
282     const correo = document.getElementById('correo').value.trim();
283     const cp = document.getElementById('codigoPostal').value.trim();
284
285     // Validacion mejorada
286     // RFC persona fisica = 4letras+6nums+3homoclave (13), moral = 3letras+6nums+3homoclave (12), poner un rcf simulado a uno real
287     const rfcRegexFisica = /^[A-Z&N]{4}\d{6}[A-Z0-9]{3}$/i;
288     const rfcRegexMoral = /^[A-Z&N]{3}\d{6}[A-Z0-9]{3}$/i;
289     const tipoCliente = document.getElementById('tipoCliente').value;
290
291     if (!nombre) errores.push('• El nombre / razón social es obligatorio.');
292     if (!rfc) errores.push('• El RFC es obligatorio.');
293     if (rfc) {
294         const rfcValido = tipoCliente === 'Persona Moral'
295             ? rfcRegexMoral.test(rfc)
296             : rfcRegexFisica.test(rfc);
297         if (!rfcValido) errores.push('• El RFC no tiene el formato correcto para ' + tipoCliente + '.');
298     }
299     if (curp && curp.length !== 18) errores.push('• La CURP debe tener exactamente 18 caracteres.');
300     // Correo es obligatorio para el sistema PLD - necesario para notificaciones
301     if (!correo) errores.push('• El correo electrónico es obligatorio.');
302     if (correo && !/^[^\s@]+@[^\s@]+\.[^\s@]+$/i.test(correo)) errores.push('• El correo electrónico no tiene un formato válido.');
303     if (tel && !/^d{10}$/i.test(tel)) errores.push('• El teléfono debe tener exactamente 10 dígitos numéricos.');
304     if (cp && !/^d{5}$/i.test(cp)) errores.push('• El código postal debe tener exactamente 5 dígitos numéricos.');
305
306     // Deshabilitar boton Siguiente mientras haya errores
307     // Esto evita que el usuario avance con datos invalidos en un sistema regulado por la CNBV
308     const btnSiguiente = document.querySelector('#paso1 .btn:not(.btn.secondary)');
309     if (errores.length > 0) {
310         mostrarErrores(errores);

```

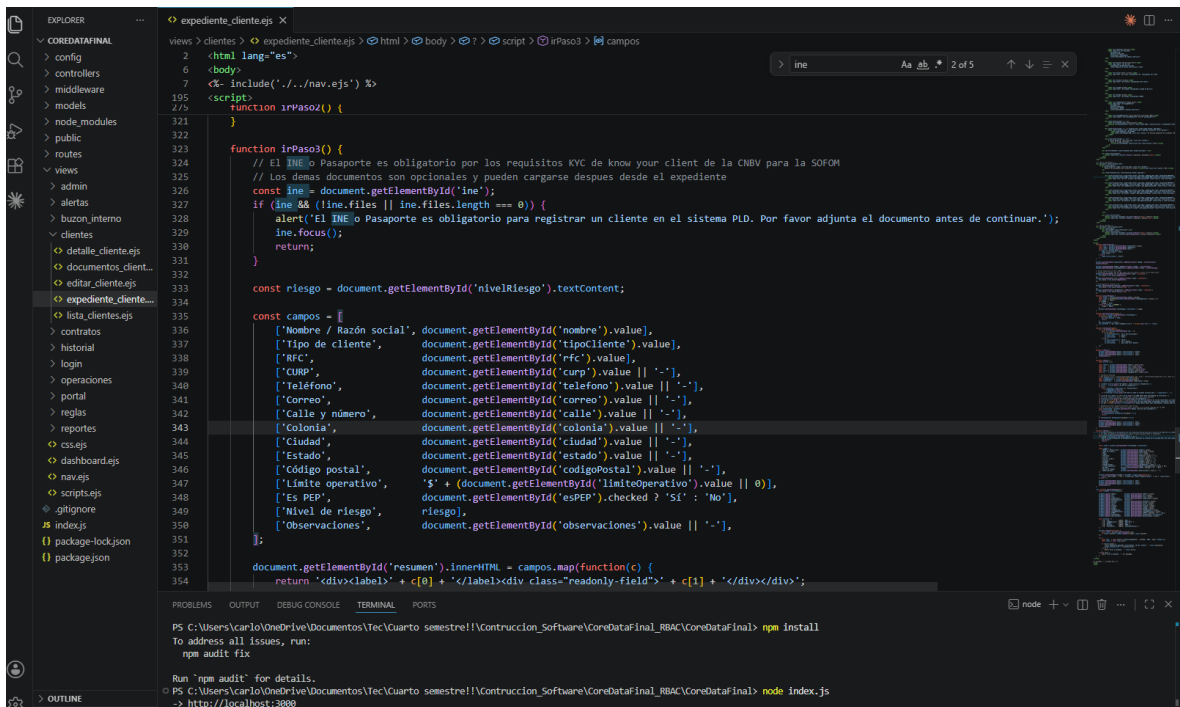
INE/Pasaporte obligatorio en paso 2\

Antes de avanzar al paso 3 se verifica que se haya seleccionado al menos el INE o Pasaporte. Los demas documentos (comprobante de domicilio, RFC, CURP, constancia fiscal) siguen siendo opcionales y pueden cargarse después desde el expediente del cliente.

```

...
function irPaso3() {
    // El INE o Pasaporte es obligatorio por los requisitos KYC de know your client
    // de la CNBV para la SOFOM
    // Los demas documentos son opcionales y pueden cargarse despues desde
    // el expediente
    const ine = document.getElementById('ine');
    if (ine && (!ine.files || ine.files.length === 0)) {
        alert('El INE o Pasaporte es obligatorio para registrar un cliente en el
        sistema PLD. Por favor adjunta el documento antes de continuar.');
        ine.focus();
        return;
    }
}

```

Buzon interno — protección de resolución por rol

RF cubierto: RF-17 Registrar reporte, RF-18 Dar seguimiento

El buzón interno anónimo o “chismógrafo” comentado por el Socio Formador es una parte importante dentro del proyecto y sistema, pues es para detectar actividad inusual dentro de la misma empresa con ayuda de testimonios anónimos. Estos no pueden existir fuera del sistema, pues sino cualquier persona con el enlace podría enviar cualquier queja o reporte al buzón, se le tiene que dejar solo para personas relacionadas a la empresa y sistema. También fue que se realizó su implementación y su creación, pero cualquier usuario podía actualizar el estatus. Implementé la protección a nivel de ruta backend y a nivel de vista frontend para que sólo el Oficial pueda gestionar reportes.

CoreDataFinal/routes/buzon_interno.routes.js

// routes/buzon_interno.routes.js

// RF-17 registrar reporte interno el Oficial + Analista

//RF 18 actualizar estatus solo Oficial de Cumplimiento

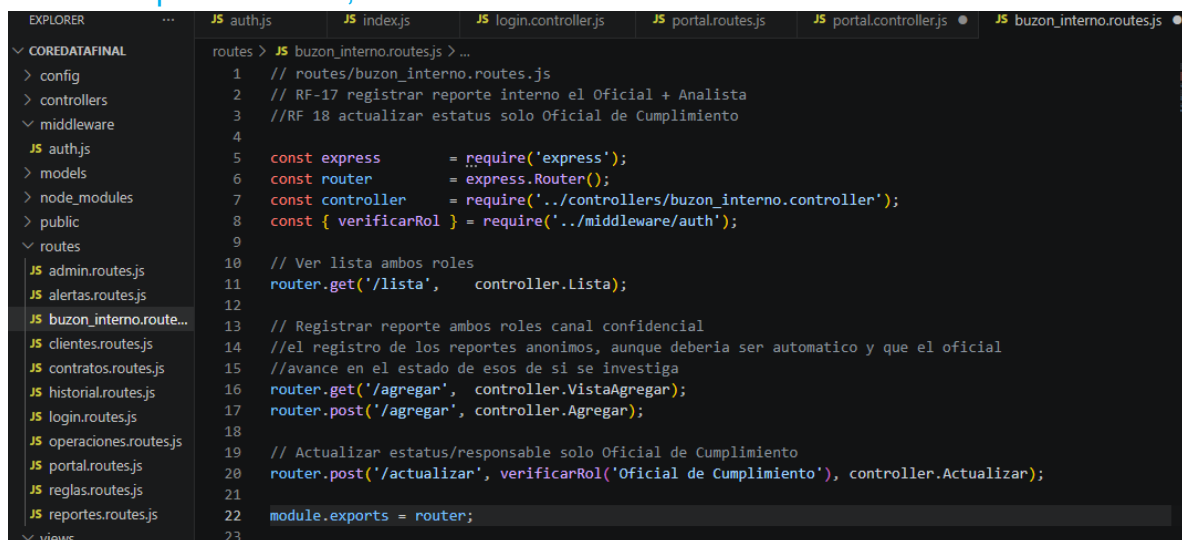
```
const express = require('express');
const router = express.Router();
const controller = require('../controllers/buzon_interno.controller');
const { verificarRol } = require('../middleware/auth');
```

```
// Ver lista ambos roles
router.get('/lista', controller.Lista);

// Registrar reporte ambos roles canal confidencial
router.get('/agregar', controller.VistaAgregar);
router.post('/agregar', controller.Agregar);

// Actualizar estatus/responsable solo Oficial de Cumplimiento
router.post('/actualizar', verificarRol('Oficial de Cumplimiento'),
controller.Actualizar);
```

```
module.exports = router;
```



```
routes > JS buzon_interno.routes.js > ...
1 // routes/buzon_interno.routes.js
2 // RF-17 registrar reporte interno el Oficial + Analista
3 //RF 18 actualizar estatus solo Oficial de Cumplimiento
4
5 const express      = require('express');
6 const router       = express.Router();
7 const controller   = require('../controllers/buzon_interno.controller');
8 const { verificarRol } = require('../middleware/auth');
9
10 // Ver lista ambos roles
11 router.get('/lista', controller.Lista);
12
13 // Registrar reporte ambos roles canal confidencial
14 //el registro de los reportes anonimos, aunque deberia ser automatico y que el oficial
15 //avance en el estado de esos de si se investiga
16 router.get('/agregar', controller.VistaAgregar);
17 router.post('/agregar', controller.Agregar);
18
19 // Actualizar estatus/responsable solo Oficial de Cumplimiento
20 router.post('/actualizar', verificarRol('Oficial de Cumplimiento'), controller.Actualizar);
21
22 module.exports = router;
23
```

Nota: El nombre del archive final puede cambiar, por ahora esta nombrado como 'CoreDataFinal'. Por lo que, pudiese cambiar de nombre, pero las rutas de los archivos permanecerán.

Consultas — RF-10, RF-14, RF-16, RF-19

RF cubierto: Consultar alertas, contratos, reportes y usuarios (Admin)

Estos 4 módulos de código siguen el mismo patrón MVC, el controller consulta Supabase y devuelve JSON limpio, la vista lo consume via fetch.

RF-10 Consultar alertas — **controllers/alertas.controller.js**

// Controller de alertas PLD automaticas

```
const modelAlertas = require('../models/alertas.model');
const modelHistorial = require('../models/historial.model');
```

```
const supabase = require('../config/supabase');

// Vista lista de alertas
module.exports.ObtenerAlertas = async (req, res) => {
  res.render('./alertas/lista_alertas', {
    mensaje: req.query.mensaje || null
  });
};

// API devuelve alertas como JSON para el frontend
module.exports.ApiListaAlertas = async (req, res) => {
  try {
    const resultado = await modelAlertas.ObtenerAlertas();
    const alertas = (resultado.alertas || []).map(function(a) {
      return {
        idAlerta: a.idalerta,
        cliente: a.idcliente || "",
        operacion: a.idoperacion || "",
        regla: a.regla || "",
        nivel: a.nivel || 'Bajo',
        fecha: a.fecha || "",
        responsable: a.responsable || "",
        estatus: a.estatus || 'Pendiente'
      };
    });
  } catch (error) {
    console.log(error);
  }
  res.json(alertas);
};
...

```

```

COREDATAFINAL
├── config
├── controllers
├── admin.controller.js
├── JS alertas.controller.js
├── JS buzon_interno.contr...
├── JS clientes.controller.js
├── JS contratos.controller.js
├── JS historial.controller.js
├── JS login.controller.js
├── JS operaciones.controll...
├── JS portal.controller.js
├── JS reglas.controller.js
├── JS reportes.controller.js
├── middleware
├── JS auth.js
├── models
├── node_modules
├── public
├── routes
├── JS admin.routes.js
├── JS alertas.routes.js
├── JS buzon_interno.route...
├── JS clientes.routes.js
├── JS contratos.routes.js
├── JS historial.routes.js
├── JS login.routes.js
├── JS operaciones.routes.js
├── JS portal.routes.js
├── JS reglas.routes.js
├── JS reportes.routes.js
├── views
├── admin
├── alertas
├── buzon_interno
├── ...

controllers > JS alertas.controller.js > ...
15 module.exports.ApiListaAlertas = async (req, res) => {
16   //
17 };
34
35
36 // API de resolver una alerta
37 module.exports.ApiResolverAlerta = async (req, res) => {
38   try {
39     const { idAlerta, resolucion } = req.body;
40     if (!idAlerta) {
41       return res.status(400).json({ msg: 'Falta idAlerta' });
42     }
43     const resultado = await modelAlertas.ResolverAlerta(idAlerta, { resolucion: resolucion || '' });
44     res.json({ exito: resultado.exito, msg: 'Alerta resuelta' });
45   } catch (error) {
46     res.status(503).json({ msg: 'Error al resolver alerta', detalle: error.message });
47   }
48 };
49
50 // Vista formulario resolver alerta
51 module.exports.VistaResolverAlerta = async (req, res) => {
52   res.render('./alertas/lista_alertas', { mensaje: null });
53 };
54
55 // Procesar resolucion de alerta
56 module.exports.ResolverAlerta = async (req, res) => {
57   res.redirect('/alertas/lista');
58 };
59
60 // API cambiar estatus de alerta (Pendiente -> En revision)
61 module.exports.ApiCambiarEstatus = async (req, res) => {
62   try {
63     const { idAlerta, estatus } = req.body;
64     const resultado = await modelAlertas.CambiarEstatus(idAlerta, estatus);
65     res.json({ exito: resultado.exito });
66   } catch (error) {
67   }
68 };

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\carlo\Downloads\CoreDataFinal_RBAC\CoreDataFinal> npm install
PS C:\Users\carlo\Downloads\CoreDataFinal_RBAC\CoreDataFinal> node index.js

```

RF-19 Gestionar usuarios — controllers/admin.controller.js

// Controlador de administracion de usuarios del sistema

```
const supabase = require('../config/supabase');
const modelPEP = require('../models/lista_pep.model');
```

// Vista lista de usuarios

```
module.exports.ListaUsuarios = async (req, res) => {
  res.render('./admin/lista_admin', {
    mensaje: req.query.mensaje || null
  });
};
```

// API devuelve usuarios como JSON para el frontend

```
module.exports.ApiListaUsuarios = async (req, res) => {
  try {
    const { data: rows, error } = await supabase
      .from('usuario')
      .select('*')
      .order('idusuario', { ascending: false });

    if (error) throw new Error(error.message);

    const usuarios = (rows || []).map(function(u) {
```

```

return {
  idUsuario: u.idusuario || u.id_usuario || u.id,
  nombre: [u.nombre || '', u.apellido || ''].join(' ').trim(),
  correo: u.correo || u.email || '',
  rol: u.rol || 'Usuario',
  estado: u.estado || 'Activo'
};
});
...

```

```

controllers > JS admin.controller.js > ApilistaUsuarios > map() callback
14 module.exports.ApilistaUsuarios = async (req, res) => {
21   if (error) throw new Error(error.message);
22
23   const usuarios = (rows || []).map(function(u) {
24     return {
25       idUsuario: u.idusuario || u.id_usuario || u.id,
26       nombre: [u.nombre || '', u.apellido || ''].join(' ').trim(),
27       correo: u.correo || u.email || '',
28       rol: u.rol || 'Usuario',
29       estado: u.estado || 'Activo'
30     };
31   });
32
33   res.json({ usuarios: usuarios });
34 } catch (error) {
35   // Tabla no creada aun - devuelven vacio sin romper el frontend
36   res.json({ usuarios: [], nota: 'Tabla usuario pendiente en Supabase' });
37 }
38 };
39
40 module.exports.VistaAgregarUsuario = async (req, res) => {
41   res.render('./admin/agregar_usuario');
42 };
43
44 module.exports.AgregarUsuario = async (req, res) => {
45   try {
46     await supabase.from('usuario').insert([
47       nombre: req.body.nombre,
48       apellido: req.body.apellido || '',
49       correo: req.body.correo,
50       rol: req.body.rol,
51       password: req.body.password,
52       activo: true
53     ]);
54   } catch (e) {}
55   res.redirect('/admin/lista');
56 };
57

```

RF-14 y RF-16 — contratos y reportes

controllers/contratos.controller.js

// Controlador de contratos

const supabase = require('./config/supabase');

// Vista lista de contratos

```

module.exports.ObtenerContratos = async (req, res) => {
  res.render('./contratos/lista_contratos', {
    mensaje: req.query.mensaje || null
  });
};

```



```
COREDATAFINAL
> config
  controllers
    JS admin.controller.js
    JS alertas.controller.js
    JS buzon_interno.contr...
    JS clientes.controller.js
    JS contratos.controller.js
    JS historial.controller.js
    JS login.controller.js
    JS operaciones.controll...
    JS portal.controller.js
    JS reglas.controller.js
    JS reportes.controller.js
  middleware
    JS auth.js
  models
  > node_modules
  > public
  > routes
    JS admin.routes.js
    JS alertas.routes.js
    JS buzon_interno.route...
    JS clientes.routes.js
    JS contratos.routes.js
    JS historial.routes.js
    JS login.routes.js
    JS operaciones.routes.js
    JS portal.routes.js
    JS reglas.routes.js
    JS reportes.routes.js
  > views
    > admin
    > alertas
    > buzon_interno
    > clientes
    > contratos
    > historial
    > login

controllers > JS contratos.controller.js > ApiListaContratos > ApiListaContratos
13 module.exports.ApiListaContratos = async (req, res) => {
22   const contratos = (rows || []).map(function(c) {
27     inicio: c.fechaInicio || c.fecha_inicio || '',
28     vencimiento: c.vencimiento || c.fecha_vencimiento || '',
29     saldo: c.saldo || 0,
30     estatus: c.estatus || ''
31   });
32   };
33   res.json({ contratos: contratos });
34   } catch (error) {
35     // Tabla no creada aun - devolver vacio sin romper el frontend
36     res.json({ contratos: [], nota: 'Tabla contrato pendiente en Supabase' });
37   }
38   };
39   };
40   };
41   module.exports.VistaAgregarContrato = async (req, res) => {
42     try {
43       const { data: clientes } = await supabase.from('cliente').select('idcliente, nombrazonsocial').order('idcliente');
44       res.render('./contratos/agregar_contrato', { clientes: clientes || [] });
45     } catch (e) {
46       res.render('./contratos/agregar_contrato', { clientes: [] });
47     }
48   };
49   };
50   module.exports.AgregarContrato = async (req, res) => {
51     try {
52       await supabase.from('contrato').insert([
53         {
54           idcliente: req.body.idcliente,
55           producto: req.body.producto,
56           fechainicio: req.body.fechainicio,
57           vencimiento: req.body.vencimiento,
58           saldo: parseFloat(req.body.saldo) || 0,
59           estatus: 'Activo'
60         }
61       ]);
62     } catch (e) {}
63     res.redirect('/contratos/lista');
64   };
65   };

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\carlo\Downloads\CoreDataFinal_RBAC\CoreDataFinal> npm install
PS C:\Users\carlo\Downloads\CoreDataFinal_RBAC\CoreDataFinal> node index.js
-> http://localhost:3000
```

Pair-programming

Durante el desarrollo del proyecto y a lo largo de los laboratorios de la unidad de formación, el equipo CoreData practicó programación en conjunto de manera constante. En el salón de clases, durante las sesiones de laboratorio, nos sentábamos juntos Luis Enrique, Luis Fernando y yo para trabajar sobre los mismos archivos, discutir decisiones de diseño y resolver problemas en tiempo real. Con los laboratorios de clase, los ejercicios o para los avances del proyecto. Cuando el tiempo de clase no era suficiente para terminar, continuábamos desde casa conectados por WhatsApp, compartiendo capturas de pantalla, mensajes de discusión y propuestas de solución, lo que funcionó como una forma de pair-programming asíncrono. Al inicio del proyecto, la colaboración fue principalmente en documentación, los primeros avances como el análisis de la solución, el diseño y la base de datos los construimos juntos, aunque en ese momento el código no se documentaba formalmente en el repositorio.

Reflexión de la competencia

Desarrollar el sistema CoreData ha sido todo un proceso de aprendizaje, que me hizo crecer significativamente como futuro desarrollador. La unidad de formación ha tenido un trabajo riguroso y constante, de laboratorios, competencias, ejercicios

y avances de proyectos semanales, todos con compromisos, responsabilidades y su peso en la materia. Con el propósito de que seamos programadores con todos los conocimientos necesarios de un Ingeniero de Software, pero no se queda allí, profundizamos en áreas de oportunidad de negocio, de trabajar un problema de la vida real y dar una propuesta de solución a ello con todo lo que se va aprendiendo semana a semana con clases y trabajos. Mi contribución más técnica y de la que más aprendí fue el RBAC que integraba casi todos los conocimientos.

Entender que la seguridad no puede vivir solo en el frontend, en esconder botones, sino que debe estar en el servidor antes de ejecutar cualquier lógica de negocio de los controladores, fue una lección concreta que aplique al diseñar verificarRol como un middleware. Cada vez que Express recibe un request, pasa por esa función antes de llegar al controller.

El portal del cliente me enseñó otros principios, entre uno de estos y que no solo es dejar espacios para subir archivos o caracteres sino que también es el aislamiento de datos. Un cliente no debe poder ver los expedientes de otro, aunque conozca la URL y se debe de poder validar los datos del cliente pues es una persona real y debe dejar registro de su existencia en el sistema para ello son los identificadores.

Trabajar en equipo en el mismo repositorio me obligo a involucrarme más al proyecto y a con mis compañeros a poder hacer los commits, hacer cambios pequeños y concretos o grandes y que se trabajen para después, documentar el código con comentarios y no mezclar múltiples módulos en un solo push, siempre cumpliendo con los principios básicos de no caer en ambigüedad, ni de repetir código.

Reflexión de Competencia – Intento 2

La entrevista como es usualmente se llevó a cabo en equipo y el reto fue desarrollar un pequeño sistema en vivo en 20 minutos y explicar lo que hicimos con la finalidad de tener el mejor aprendizaje posible de la Unidad de Formación. Al presentarnos como equipo, llegamos sin una estructura preparada de antemano. Podíamos llegar con algo hecho y adaptarlo al problema que se nos proporcionara para resolver, pero no lo hicimos. Eso fue el error principal y lo más honesto que puedo decir al respecto, fuimos a la entrevista en blanco, ahora sabemos con mas claridad lo que ya conocíamos, el código toma tiempo, una estructura clara que fuera adaptable y que pudiéramos desarrollar por lo menos una funcionalidad a la resolución del problema era mas que suficiente para la acreditación total de la competencia, pues lo siento como una entrevista de trabajo, donde se tiene un problema y se debe

desarrollar con lógica y conocimiento alguna funcionalidad o solución, para esta competencia debimos estar más preparados.

En esos 20 minutos de competencia, yo logré escribir el archivo “database.js” proporcionado en la carpeta de competencia, con los datos base y las funciones getTodos y getPorId, que sería la capa de modelo en MVC. Sin embargo, sin el resto de la estructura, ese código no demuestra el dominio completo de la competencia porque no llega a integrarse con un controlador ni con una vista. Sé cómo hacerlo, lo hice en el proyecto CoreData, pero en la entrevista no tuve tiempo de mostrarlo, nuevamente recalcando el deber y poder de haber utilizado una estructura previa.

El segundo problema fue la colaboración en vivo del equipo. Intentamos crear un repositorio en GitHub durante la entrevista para trabajar en equipo y tuvimos problemas con las invitaciones para colaborar, con los permisos para hacer “git push y git pull” y todo eso consume tiempo que no teníamos. En el proyecto real ya tenemos eso resuelto porque el repositorio y los permisos están configurados desde semanas antes. Lo que nos faltó fue trasladar esa misma preparación al contexto de la entrevista.

Lo que aprendí de esto es que el dominio de una competencia no solo se mide en código que funciona, sino en la capacidad de demostrarlo bajo presión y en un tiempo limitado. Un desarrollador con dominio real llega a ese tipo de sesión con una estructura mínima lista, carpetas organizadas, un index.js funcional básico, el modelo, el controlador y la ruta ya conectados, para que cualquier problema que le den solo requiera adaptar la lógica, no construir desde cero. Eso es lo que nos faltó y es la diferencia entre saber hacerlo y poder demostrarlo cuando importa. Pues hemos aprendido durante clases que deberemos trabajar en códigos de desarrolladores predecesores o crear nuestro código, pero unas cuantas líneas de código no demostraran capacidad de resolver a necesidades reales de los clientes o generar valor a las empresas, todo es un proceso integrativo en el que se debe de adaptar y estar en constante crecimiento, con organización, documentación, metodología, gestión de la configuración y resolución.

Referencias

Anthropic. (2026). Asistencia en el diseño e implementación del módulo RBAC para sistema Node.js/Express [Conversación con modelo de IA Claude Sonnet 4.5]. Claude.ai. <https://claude.ai>

Anthropic. (2026). Asistencia en la implementación de validaciones de formulario para registro de clientes PLD/FT [Conversación con modelo de IA Claude Sonnet 4.5]. Claude.ai. <https://claude.ai>

Anthropic. (2026). *Apoyo en el desarrollo del sistema CoreData Consulting — módulos de alertas, operaciones, buzón interno y pruebas* [Conversación con modelo de IA Claude Sonnet 4.5]. Claude.ai. <https://claude.ai>

Anthropic. (2026). Asistencia en la implementación de validaciones de formulario: RFC con expresiones regulares, correo obligatorio y botón deshabilitado durante errores [Conversación con modelo de IA Claude Sonnet 4.6]. Claude.ai. <https://claude.ai>

Nielsen, J. (1994). 10 Usability Heuristics for User Interface Design. Nielsen Norman Group. <https://www.nngroup.com/articles/ten-usability-heuristics/>

Mozilla Developer Network. (2024). Regular expressions — JavaScript. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions

SAT. (2024). Estructura del RFC para personas físicas y morales. Servicio de Administración Tributaria. <https://www.sat.gob.mx>

Anthropic. (2026). Utilizado en apoyo y construcción, referenciado en los avances oficiales del equipo CoreData. Claude.ai. <https://claude.ai>